

```

#include<iostream>
using namespace std;
struct Node {
    float marks = 0;
    Node* next = nullptr;
};
class SCLL {
    long list_size = 0;
    Node* starting_node = nullptr;
public:
    SCLL(void) {}
    ~SCLL(void);
    void print(void);
    void push_front(float marks_to_add);
    void push_back(float marks_to_add);
    void delete_kth(long index);
    float at(long index);
};

float SCLL::at(long index) {
    if (index < 1 || index > list_size) {
        return -1;
    }
    Node* current_node = starting_node;
    for (long i = 1; i < index; i++) {
        current_node = current_node->next;
    }
    return current_node->marks;
}

void SCLL::delete_kth(long index) {
    if (index < 1 || index > list_size) {
        return;
    }
    if (index == 1) {
        Node* temp = starting_node;
        starting_node = starting_node->next;
        delete temp;
    }
    else {
        Node* previous_node = nullptr;
        Node* current_node = starting_node;
        for (long i = 1; i < index; i++) {
            previous_node = current_node;
            current_node = current_node->next;
        }
        previous_node->next = current_node->next;
        delete current_node;
    }
    list_size--;
}

void SCLL::push_back(float marks_to_add) {
    Node* new_node = new Node;
    new_node->marks = marks_to_add;
    new_node->next = nullptr;
    if (list_size == 0) {
        starting_node = new_node;
        starting_node->next = starting_node;
    }
}

```

```

        else {
            Node* current_node = starting_node;
            while (current_node->next != starting_node) {
                current_node = current_node->next;
            }
            current_node->next = new_node;
            new_node->next = starting_node;
        }
        list_size++;
    }
}

SCLL::~SCLL(void) {
    Node* current_node = starting_node;
    while (list_size > 0) {
        Node* next_node = current_node->next;
        cout << "Deleting: " << current_node->marks << endl;
        delete current_node;
        current_node = next_node;
        list_size--;
    }
}

void SCLL::print(void) {
    cout << endl << "Printing list...." << endl;
    if (list_size == 0) {
        cout << "The list is empty!!" << endl;
    }
    else {
        Node* current_node = starting_node;
        do {
            cout << current_node->marks << ", ";
            current_node = current_node->next;
        } while (current_node != starting_node);
        cout << endl;
    }
    cout << endl;
}

void SCLL::push_front(float marks_to_add) {
    cout << "Adding to front: " << marks_to_add << endl;
    Node* new_node = new Node;
    new_node->marks = marks_to_add;
    new_node->next = nullptr;
    if (list_size == 0) {
        starting_node = new_node;
        starting_node->next = starting_node;
    }
    else {
        Node* current_node = starting_node;
        while (current_node->next != starting_node) {
            current_node = current_node->next;
        }
        new_node->next = starting_node;
        starting_node = new_node;
        current_node->next = starting_node;
    }
    list_size++;
}
}

```

```
int main() {
    SCLL L;
    L.print();
    L.push_front(50);
    L.push_front(40);
    L.push_front(30);
    L.push_front(20);
    L.push_front(10);
    L.push_back(60);
    L.push_back(70);
    L.push_back(80);
    L.push_back(90);
    L.push_back(100);
    L.print();
    cout << endl << " Marks at index 3: " << L.at(3) << endl << endl;
    cout << " Deleting marks at index 3: " << L.at(3) << endl;
    L.delete_kth(3);
    L.print();
    return 0;
}
```